Assignment 4: Parallelization
Galaxy Simulation (Pthreads and OpenMP)

Sotirios Oikonomou

February 25, 2026

1 The Problem

The galaxy simulation computes gravitational interaction between N particles. At each timestep, every particle interacts with every other particle, resulting in $\mathcal{O}(N^2)$ force computation [4, 5].

The objective is to parallelize the dominant force computation using Pthreads while maintaining correctness and evaluating scalability [1].

2 The Solution

2.1 Data structures

Particle data is stored in a flat array of size $6N$:

$$\texttt{data}[6i + k], \quad k \in \{X, Y, M, VX, VY, B\}$$

Accelerations are stored in:

- Global arrays `ax[N]` and `ay[N]`
- Thread-private arrays `ax_priv[p*N]` and `ay_priv[p*N]`

2.2 Serial Optimization

Before introducing parallelization, the serial implementation of the galaxy simulation was carefully optimized. According to Amdahl's Law [6], the achievable parallel speedup is limited by the efficiency of the sequential baseline. Therefore, it is essential that the serial version is already computationally efficient and well structured [1].

Baseline algorithm and complexity

The simulation computes gravitational interactions between all particle pairs. A naïve implementation evaluates all ordered pairs (i, j), resulting in $\mathcal{O}(N^2)$ complexity per timestep [4, 5]. The velocity and position updates are $\mathcal{O}(N)$ and therefore asymptotically negligible compared to the force computation [5].

This confirms that the force calculation loop is the dominant computational component of the algorithm [4].

Data layout and memory locality

All particle data is stored in a single contiguous array:

```
double *data = malloc(6*N*sizeof(double));
```

Each particle occupies six consecutive doubles: position (x, y), velocity (v_x, v_y), mass, and brightness. Access is performed using indexed offsets such as:

```
data[6*i + X],  data[6*i + Y].
```

This layout ensures:

- Contiguous memory access,
- Good spatial locality,
- Efficient cache utilization,
- Suitability for compiler optimizations.

Keeping particle data compact in memory reduces cache misses during the $\mathcal{O}(N^2)$ force computation [9].

Separation of force and update phases

Accelerations are stored in separate arrays:

```
double *ax, *ay;
```

For each timestep:

1. All accelerations are first computed.
2. Velocities and positions are updated afterward.

This separation ensures:

- Correct symplectic Euler integration,
- No mixing of old and newly updated values,
- A clean computational structure (force phase $\rightarrow$ update phase).

This phase separation later simplifies the parallelization, where synchronization naturally occurs between phases [1].

Symmetry optimization (Newton's third law)

The most significant serial optimization exploits symmetry in pairwise interactions.

Instead of computing both (i, j) and (j, i), the nested loop is written as:

```
for (i = 0; i < N; i++)
    for (j = i+1; j < N; j++)
```

For each pair, the interaction is computed once, and both particles are updated using equal and opposite forces (Newton's third law) [2]. This:

- Eliminates redundant computations,
- Reduces the number of force evaluations by approximately half,
- Preserves physical correctness.

This optimization significantly improves serial runtime without altering the physical model [4].

Compiler optimizations

The code was compiled using:

```
-O3 -march=native
```

The `-O3` flag enables aggressive compiler optimizations such as loop unrolling, instruction scheduling, and vectorization (where possible). The `-march=native` flag enables architecture-specific instruction usage [11].

All reported performance measurements were obtained with compiler optimizations enabled.

Timing-based hotspot identification

Before parallelization, the program was instrumented using `gettimeofday()` to measure separately:

- Total simulation time,
- Force computation time,
- Update phase time.

Measurements were performed with graphics disabled to avoid distortion of timing results [1].

For $N = 5000$, 100 timesteps, and $\Delta t = 10^{-5}$, the measured runtime was approximately:

- Total time: ≈ 3.3 s,
- Force computation: $> 99\%$ of total time,
- Update phase: negligible.

This confirms that the force computation completely dominates runtime, and therefore it is the natural and necessary target for parallelization [1, 6].

Measured effect of symmetry optimization

To quantify the impact of the symmetry optimization (Newton's third law), performance measurements from Assignment 3 were reused. The naive implementation computes all ordered pairs (i, j), whereas the optimized version computes only pairs with $j > i$ and updates both particles from a single interaction.

Table 1 summarizes the measured user times for 100 timesteps with graphics disabled and compiler flags `-O3 -march=native`.

N	Naïve (s)	Symmetry (s)	Improvement (%)
1000	0.146	0.122	16.4
2000	0.592	0.454	23.3
3000	1.316	1.140	13.4

Table 1: Measured runtime reduction from symmetry optimization (O3 build, 100 steps, graphics disabled).

The symmetry optimization reduces the number of pairwise force evaluations by approximately half. In practice, the measured runtime improvement ranges between 13% and 23%, depending on problem size. The gain is smaller than the theoretical factor of two because memory accesses, loop overhead, and the $\mathcal{O}(N)$ update phase still contribute to total runtime. Nevertheless, this optimization substantially reduces the dominant force computation cost and forms the basis for efficient parallelization.

2.3 Parallelization with Pthreads

Particles are partitioned into contiguous blocks:

$$\text{chunk} = \left\lceil \frac{N}{p} \right\rceil$$

Each timestep consists of three phases:

1. **Force phase**: Each thread computes pairwise interactions for its particle range using Newton's third law (each pair computed once with $j > i$) [2].
2. **Reduction phase**: Thread-private accelerations are reduced into global arrays.
3. **Update phase**: Velocities and positions are updated using symplectic Euler [3].

Private slices avoid race conditions without locks or atomics [7].
Threads are created and joined for each phase and timestep [7].

2.4 Parallelization with OpenMP

The OpenMP implementation follows the same computational structure as the Pthreads version but uses compiler directives instead of explicit thread management [8].

A single persistent parallel region is created:

```
#pragma omp parallel
{
    for (step = 0; step < nsteps; step++) {
        ...
    }
}
```

This avoids the thread creation and destruction overhead that is present in the Pthreads implementation [8].

Inside the parallel region, the force computation is parallelized using:

```
#pragma omp for schedule(static)
for (int i = 0; i < N; i++)
```

Each thread processes a subset of particles in a static partitioning scheme [8]. As in the Pthreads implementation, thread-private acceleration arrays (`ax_priv`, `ay_priv`) are used to avoid race conditions when applying Newton's third law updates to both particles in a pair [2, 8].

After the force phase, a manual reduction is performed to accumulate private accelerations into the global arrays (`ax`, `ay`). Implicit barriers at the end of OpenMP work-sharing constructs ensure correct synchronization between phases [8].

Timing alignment is handled using `#pragma omp barrier` and `#pragma omp master` to measure the force phase accurately [8].

Alternative parallelization options

Several alternative approaches were considered. A direct shared update of the global acceleration arrays using `pthread_mutex` locks (e.g., one lock per particle) was rejected because the force phase performs $\Theta(N^2)$ pair updates, which would create extreme lock contention and high synchronization overhead. Using atomic operations for each acceleration update was also avoided for the same reason: atomics would be executed at very high frequency and would substantially reduce scalability.

Another option is to compute all ordered pairs (i, j) independently (dropping Newton's third law symmetry) so that each thread only updates particle i and never touches j. This eliminates race conditions, but doubles the arithmetic work, increasing runtime and memory traffic, which is undesirable given that the force phase already dominates execution time.

Finally, dynamic scheduling in OpenMP was considered but not used. With the chosen loop structure and static partitioning, the workload is already well balanced while static scheduling keeps overhead minimal. In practice, runtime is limited primarily by memory bandwidth and the reduction cost rather than by load imbalance, so dynamic scheduling would not provide a clear benefit.

3 Performance and Discussion

3.1 Reproducibility and measurement methodology

All performance experiments were conducted on the following system:

- CPU model: Apple M4 Pro
- Number of physical cores: 14 (10 Performance and 4 Efficiency)
- Logical cores: 14
- OS: ProductName: macOS, ProductVersion: 26.3, BuildVersion: 25D125
- Compiler: Apple clang version 17.0.0 (clang-1700.6.3.2)

The code was compiled using:

```
make fast_pthreads
```

which corresponds to the compilation flags:

```
-O3 -march=native -Wall -Wextra -std=c11 -pthread -lm
```

The program was executed using:

```
./galsim_p N input_data/ellipse_N_XXXXX.gal 100 1e-5 0 p
```

where $p \in \{1, 2, 4, 8\}$ is the number of threads. All timings were measured with graphics disabled (graphics = 0).

The OpenMP version was compiled using:

```
make fast_openmp
```

with flags including `-fopenmp` and linking against `libomp` on macOS [12].

The OpenMP program was executed using:

```
./galsim_omp N input_data/ellipse_N_XXXXX.gal 100 1e-5 0 p
```

where $p \in \{1, 2, 4, 8\}$ is the number of OpenMP threads.

Wall-clock time was measured using `gettimeofday()` via the function `get_wall_seconds()`, and the reported value is the total runtime [10].

Each configuration was run multiple times and the minimum total time was used for performance evaluation [1].

Speedup is defined as:

$$S(p) = \frac{T(1)}{T(p)}, \qquad E(p) = \frac{S(p)}{p}$$

[6]

3.2 Results for N = 5000

Threads	Time (s)	Speedup	Efficiency
1	3.286 999	1.000	1.000
2	2.509 061	1.310	0.655
4	1.398 213	2.351	0.588
8	0.731 430	4.492	0.561

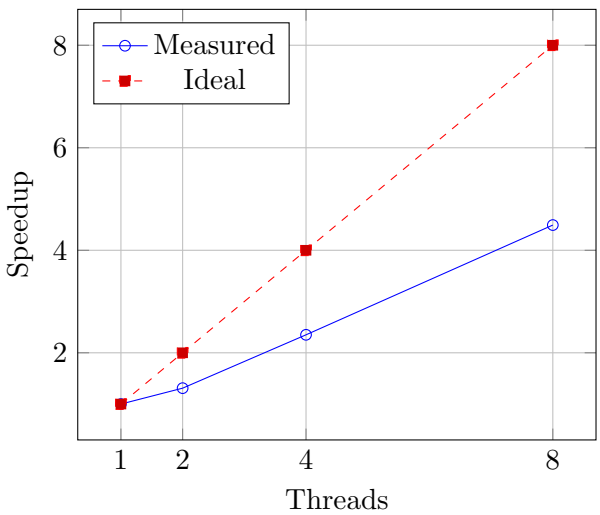

Figure 1: Speedup for $N = 5000$.

3.3 Results for N = 10000

Threads	Time (s)	Speedup	Efficiency
1	12.403 920	1.000	1.000
2	9.411 905	1.318	0.659
4	5.193 996	2.388	0.597
8	2.742 000	4.524	0.566

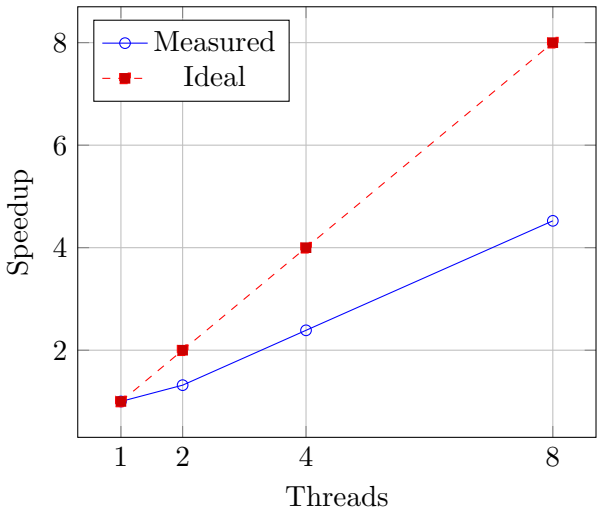

Figure 2: Speedup for $N = 10000$.

3.4 OpenMP Speedup for N = 5000

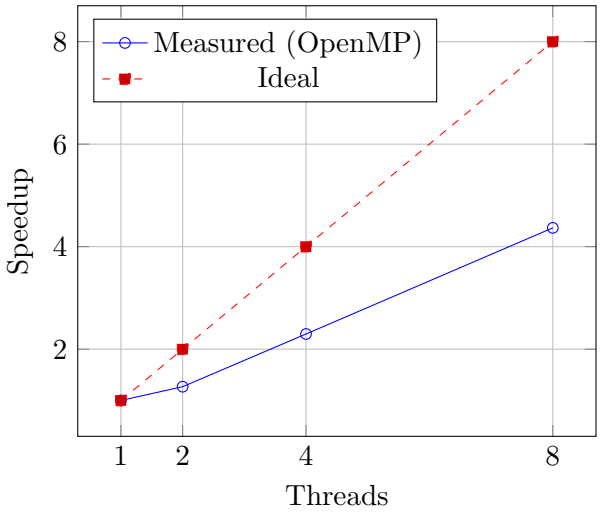

Figure 3: OpenMP speedup for $N = 5000$ (times: 3.300780, 2.602342, 1.437742, 0.755826 s).

3.5 OpenMP Speedup for N = 10000

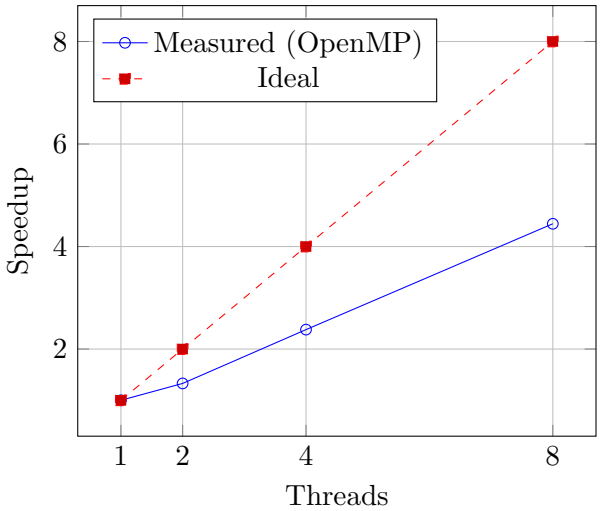

Figure 4: OpenMP speedup for $N = 10000$ (times: 12.730741, 9.578308, 5.351836, 2.864861 s).

3.6 Scaling behaviour as a function of N

In addition to speedup plots, we examine how runtime scales with problem size N. The figure below shows total runtime for 1 thread (serial baseline) and 8 threads.

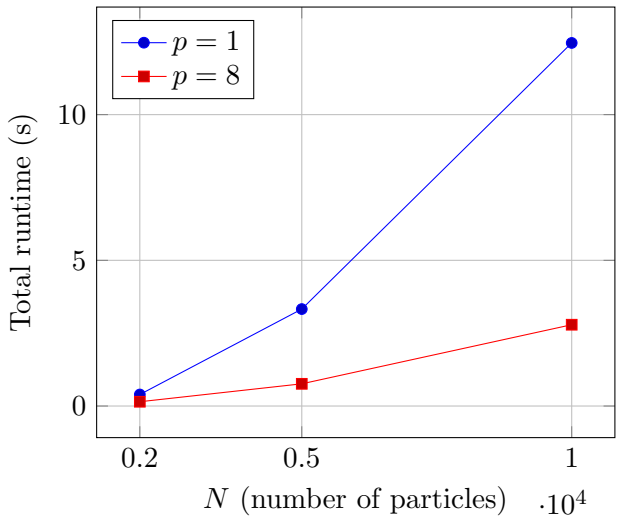

Figure 5: Runtime scaling with problem size N for 1 and 8 threads.

The $N = 20000$ case is omitted because the corresponding input file was not available. The quadratic growth in runtime for the serial case is consistent with the $\mathcal{O}(N^2)$ complexity of the force computation [4, 5]. Parallel execution significantly reduces absolute runtime, but preserves the same asymptotic behaviour [6].

3.7 Discussion

The force computation accounts for more than 99% of the runtime, confirming that it is the dominant bottleneck [6].

Speedup increases with thread count but remains below ideal linear scaling. At 8 threads the efficiency is approximately 0.56–0.57 for both problem sizes [6].

The limiting factors include:

- Pthreads thread creation/join overhead due to launching threads for each phase and timestep [7].
- Memory bandwidth limitations in the $\mathcal{O}(N^2)$ loop [9].
- Reduction overhead proportional to pN [8].

It is generally expected that larger problem sizes achieve speedup closer to the ideal case because the computation-to-overhead ratio increases [6]. In our experiments, the larger problem size ($N = 10000$) shows slightly improved speedup for lower thread counts compared to $N = 5000$, which is consistent with this expectation. However, at 8 threads the efficiencies are very similar for both sizes, indicating that memory bandwidth and synchronization overhead limit scalability for higher thread counts [9]. When comparing Pthreads and OpenMP, both implementations achieve very similar scalability and efficiency [7, 8]. The Pthreads version incurs explicit thread creation and join overhead at every phase, while the OpenMP version uses a persistent parallel region, reducing management overhead [8]. However, the overall speedup is primarily limited by memory bandwidth and the $\mathcal{O}(pN)$ reduction cost rather than the threading model itself [8, 9].

References

[1] High Performance Programming, Uppsala University, *Assignment 3: The Gravitational N-body Problem*, Spring 2022.

[2] I. Newton, *Philosophiæ Naturalis Principia Mathematica*, 1687.

[3] E. Hairer, C. Lubich, and G. Wanner, *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*, Springer Series in Computational Mathematics, 2nd ed., Springer, 2006.

[4] S. J. Aarseth, *Gravitational N-Body Simulations: Tools and Algorithms*, Cambridge University Press, 2003.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.

[6] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *AFIPS Conference Proceedings*, vol. 30, pp. 483–485, 1967.

[7] D. R. Butenhof, *Programming with POSIX Threads*, Addison-Wesley, 1997.

[8] OpenMP Architecture Review Board, *OpenMP Application Programming Interface Specification.*

[9] G. Hager and G. Wellein, *Introduction to High Performance Computing for Scientists and Engineers*, CRC Press, 2010.

[10] The Open Group, *The Open Group Base Specifications, Issue 7 (POSIX.1-2017)*, `gettimeofday()`.

[11] LLVM Project, *Clang User Manual* (compiler optimization options).

[12] Apple/LLVM toolchain notes for OpenMP usage on macOS (linking against `libomp`).